

← Go Back

Hardware

GIGA R1 WiFi

Tutorials

Arduino GIGA R1 User Manual

Guide to GIGA R1 Advanced ADC/DAC and Audio Features

GIGA R1 Camera Guide

Dual Core Debugging the GIGA R1 WiFi

Guide to GIGA R1 Dual Cores

Getting Started with GIGA R1 WiFi

MicroPython on the GIGA R1

Guide to Arduino GIGA USB Features

GIGA R1 WiFi Network Examples

Home / Hardware / GIGA R1 WiFi / Guide to GIGA R1 Advanced ADC/DAC and Audio Features

Guide to GIGA R1 Advanced ADC/DAC and Audio Features

Learn how to use the ADC/DAC features, along with useful examples on how to generate waveforms and play audio from a file.

Author José Bagur, Taddy Chung & Karl Söderby Last revision 09/26/2024

In the GIGA R1, you can find the powerful STM32H747XI, a dual-core 32-bit Arm® Cortex® microcontroller from STMicroelectronics; this is the same microcontroller found in the Portenta H7 board.

In this guide, we will focus on the advanced ADC/DAC features, utilizing the Arduino_AdvancedAnalog library. The examples found in this guide can be used to:

Set up and read ADCs with specific parameters (resolution, sample rate, number of samples per channel, queue depth).

Set up and write to a DAC channel with specific parameters (resolution, frequency, number of samples per channel, queue

ON THIS PAGE

Hardware & Software Needed

- ADC/DAC Pins and Connectors
- Analog-to-Digital Converter (ADC) +
- Digital-to-Analog Converters (DAC) +
- Audio Playback +
- Pulse Density Modulation Support

Generate specific waveforms through input via serial commands (triangle, square, sine, sawtooth waves) as well as adjusting the frequency.

Read and play audio files (`.wav`) from a USB stick (connected to USB-A) to a speaker, using the audio jack.

Important note: the GIGA R1 does NOT have an amplifying circuit onboard.

 Connecting speakers that does not have an amplifier can damage the DAC and the board itself.

Hardware & Software Needed

GIGA R1 WiFi.

Arduino IDE

Arduino_AdvancedAnalog library.

And for specific examples:

USB Mass Storage Device (USB Stick).

Auxiliary Cable.

Speaker with a built-in amplifier.

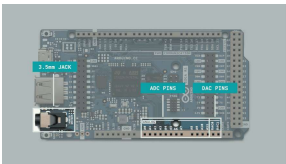
ADC/DAC Pins and Connectors

The GIGA gives you access to more pins than any other Arduino board accessible for makers.

audio features or can be used for developing audio applications. Audio pins and connectors in the GIGA can be divided into three important groups:

- Analog-to-Digital Converters (ADC) pins
- Digital-to-Analog Converters (DAC) pins
- Tip, Ring, Ring, Sleeve (TRRS) 3.5mm jack

The image below shows the position of the audio pins and connectors of the GIGA R1:



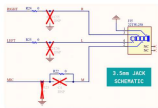
ADC/DAC pins and connectors of the GIGA R1

The table below explains the full functionality of the listed on it; notice that A12 and A13 can be used as DACs (DAC0, DAC1).

Pin	Functionality
A0	ADC
A1	ADC
A2	ADC
A3	ADC
A4	ADC
A5	ADC
A6	ADC
A7	ADC
A8	ADC
A9	ADC

Pin	Functionality
A11	ADC
A12 / DAC0	ADC and DAC
A13 / DAC1	ADC and DAC

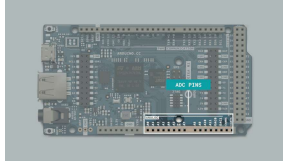
Pins `A7`, `DAC0`, and `DAC1` can also be accessed via the built-in TRRS 3.5mm jack. `DAC0` is connected to ring 1 (right channel), `DAC1` is connected to the tip (left channel), and `A7` is connected to ring 2 (microphone) of the jack, as shown in the schematic below:



GIGA R1 TRRS 3.5mm
jack schematic

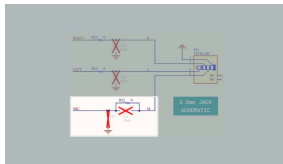
Analog-to-Digital Converter (ADC)

An analog-to-digital converter (ADC) is a device that converts an analog voltage, or signal, into digital data. The GIGA R1 microcontroller, the STM32H747XI, embeds three ADCs whose resolution can be configured to 8, 10, 12, 14, or 16 bits. Each ADC shares up to 20 external channels that can be accessed in the GIGA R1 board through pins `A0`, `A1`, `A2`, `A3`, `A4`, `A5`, `A6`, `A7`, `A8`, `A9`, `A10`, and `A11`; pins `DAC0`, `DAC1` can also be used as ADCs.



ADC pins of the GIGA
R1

A7 is connected to ring 2 (microphone) of the jack, as shown in the schematic below:



GIGA R1 3.5mm input
jack schematic

The GIGA R1 ADCs can be used with the [built-in analog input/output functions of the Arduino programming language](#), though they only provide the basic functionalities of the ADCs. To use all of the capabilities of the DACs from the GIGA R1, we can use the [Arduino_AdvancedAnalog](#) library from Arduino. Let's check some interesting examples that show some capabilities of the GIGA R1 ADCs!

Multi Channel ADC

The following example code show how to use two GIGA R1 ADCs simultaneously with the [Arduino_AdvancedAnalog](#) library from Arduino:

```
1 // This example shows
2 #include <Arduino_Adv
3
4 AdvancedADC adc1(A0);
5 AdvancedADC adc2(A1);
6 uint64_t last_millis
7
8 void setup() {
9     Serial.begin(9600
10
11     // Resolution, sa
12     if (!adc1.begin(A
13         Serial.printl
14         while (1);
15     }
16
17     if (!adc2.begin(A
18         Serial.printl
19         while (1);
20     }
21 }
22
23 void adc_print_buf(Ad
24     if (adc.available
25         SampleBuffer
26
27         // Print firs
28         Serial.printl
```

ADC Serial Plotter Example

The following example code shows how to use two GIGA R1 ADCs simultaneously with the **Arduino_AdvancedAnalog** library from Arduino and displays the readings via the Serial Plotter of the Arduino IDE:

```
1  #include <Arduino_Adv
2
3  AdvancedADC adc(A0, A
4  uint64_t last_millis
5
6  void setup() {
7      Serial.begin(9600
8
9      // Resolution, sa
10     if (!adc.begin(AN
11         Serial.printl
12         while (1);
13     }
14 }
15
16 void loop() {
17     if (adc.available
18         SampleBuffer
19         // Process th
20         if ((millis()
21             Serial.prin
22             Serial.prin
23             last_millis
24         }
25         // Release th
26         buf.release()
27     }
28 }
```

Digital-to-Analog Converters (DAC)

A digital-to-analog converter (DAC) is a device that has a function opposite to that of the analog-to-digital converter (ADC); a DAC converts digital data to an analog voltage. The GIGA R1 microcontroller, the STM32H747XI, features two 12-bit buffered DAC channels that can convert two digital signals into two analog voltage signals. Some of the features of the DACs found in the GIGA R1 are the following:

Left or right data alignment
in 12-bit mode

Dual DAC channel
independent or
simultaneous conversions

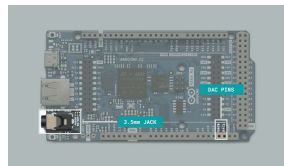
DMA capability for each
channel

External triggers for
conversion

Input voltage reference or
internal voltage reference

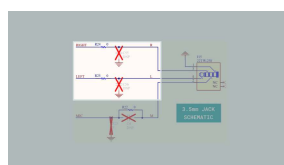
Analog waveform
generation

The GIGA R1 DACs are named `DAC0` and `DAC1`; they can be found on pins `A12` and `A13` correspondingly, as shown in the image below:



DAC0, DAC1 and the
3.5mm input jack of the
GIGA R1

Besides pins `A12` and `A13`, `DAC0` and `DAC1` can also be accessed via the built-in TRRS 3.5mm jack. `DAC0` is connected to the right channel (tip), while `DAC1` is connected to the left channel (ring) of the input jack as shown in the schematic below:



GIGA R1 3.5mm input
jack schematic

Waveform Generation with the GIGA R1 DACs

Waveform generation is an exciting application used in audio systems, for example, synthesizers, for audio signal generation.

The following example shows how to output an 8kHz square wave on

DAC0:

```
1 // This example output
2 #include <Arduino_Adv
3
4 AdvancedDAC dac0(A12)
5
6 void setup() {
7     Serial.begin(9600
8
9     while (!Serial) {
10
11     }
12
13     if (!dac0.begin(A
14         Serial.printl
15         while (1);
16     }
17 }
18
19 void dac_output_sq(Ad
20     if (dac_out.avail
21
22         // Get a free
23         SampleBuffer
24
25         // Write data
26         for (int i=0;
27             buf.data(
28         }
```

DAC Multi Channel Square Waves

It is also possible to

The following example generates an 8kHz square wave on `DAC0`, while a 16kHz square wave is generated on `DAC1`:

```
1 // This example output
2 #include <Arduino_Adv
3
4 AdvancedDAC dac0(A12)
5 AdvancedDAC dac1(A13)
6
7 void setup() {
8     Serial.begin(9600
9
10    while (!Serial) {
11
12    }
13
14    if (!dac0.begin(A
15        Serial.printl
16        while (1);
17    }
18
19    if (!dac1.begin(A
20        Serial.printl
21        while (1);
22    }
23 }
24
25 void dac_output_sq(Ad
26     if (dac_out.avail
27
28     // Get a free
```

DAC Sine Wave

A 32kHz sine wave output on `DAC0` can be generated using the following example:

```
1 // This example output
2 #include <Arduino_Adv
3
4 AdvancedDAC dac0(A12)
5
6 uint16_t lut[] = {
7     0x0800, 0x08c8, 0x0
8     0x0fff, 0x0ff5, 0x0
9     0x0800, 0x0737, 0x0
10    0x0000, 0x000a, 0x0
11 };
12
13 static size_t lut_siz
14
15 void setup() {
16     Serial.begin(9600
17     if (!dac0.begin(A
18         Serial.printl
19         while (1);
20     }
21 }
22
23 void loop() {
24     static size_t lut
25
26     if (dac0.availabl
27         // Get a free
28         SampleBuffer
```

Waveform Generation Based on Input

The following example allows you to switch between several waveforms via a serial interface. Uploading the example will allow you to change it based on the character input.

- ☐ t : generate triangle wave
- ☐ q : square wave
- ☐ s : sine wave
- ☐ r : sawtooth wave
- ☐ + : increase frequency
- ☐ - : decrease frequency

```
1 // This example gene
2
3 #include <Arduino_Ac
4
5 #define N_SAMPLES
6 #define DEFAULT_FREQ
7
8 AdvancedDAC dac0(A12
9 uint8_t SAMPLES_BUFF
10 size_t dac_frequency
11
12 void generate_wavefo
13 {
14     switch (cmd) {
15         case 't':
16             // Trian
17             Serial.p
18             for (int
19                 SAMP
20             }
21             break;
22
23         case 'q':
24             // Squar
25             Serial.p
26             for (int
27                 SAMP
28             }
```

Audio Playback

The GIGA R1 12-bit DAC channels can also be used to read `.wav` files from a USB stick and stream them directly to a speaker.

For this example, you will need:

- A speaker, that has a built in amplifier.

- A USB mass storage device (USB stick).*

- [Arduino_USBHostMbed5](#) library installed.

*USB mass storage devices connected needs to be formatted with the FAT32 as a file system, using the MBR partitioning scheme. Read more in the [USB Mass Storage](#) section.

USB Stick Configuration

The **Arduino_AdvancedAnalog** library contains the necessary functions that enable us to use the advanced capabilities of the GIGA R1 DACs.

To read `.wav` files from the USB stick we are using the **Arduino_USBHostMbed5** library. It is important that the USB stick is formatted properly, and that we define its name in the sketch. In this case, we name it `USB_DRIVE`, and is defined like this:

```
1 mbed::FATFileSystem usb
```

Another line of code defines the audio file to be played. Here we need to write the name of the `.wav` file we want to play, but also the directory (which is `/USB_DRIVE` in this case).

```
1 file = fopen("/USB_DRIV
```

Play Single Audio File

The following example plays a single `.wav` file named `AUDIO_SAMPLE.wav` from a USB mass storage device named `USB_DRIVE`.

The file is read, processed and writes the values to a buffer, which is then outputted via the DAC0 channel. The DAC0 channel is connected to the 3.5mm audio jack, and when connecting it to a speaker, it will play the file **once**.

Note that to start the sketch, you need to open Serial Monitor due to the `while(!Serial)` blocker in the `setup()`. Information on the operation is printed in the monitor, as well as errors if the operation fails.

```

1  /*
2   * GIGA R1 - Audio P
3   * Simple wav format
4   * In order for this
5   * Furthermore you n
6  */
7
8  #include <Arduino_Ac
9  #include <DigitalOut
10 #include <Arduino_US
11 #include <FATFileSys
12
13 AdvancedDAC dac0(A12
14
15 USBHostMSD msd;
16 mbed::FATFileSystem
17
18 FILE * file = nullpt
19 int sample_size = 0;
20 int samples_count =
21
22
23 void setup()
24 {
25     Serial.begin(11520
26     while (!Serial);
27
28     /* Enable power fo

```

Loop Multiple Audio Files

This example is similar to the **Play Single Audio File** example, but with some key changes:

This example uses multiple audio files.

The file read is moved to a separate function, `configFile()`, as it will be continuously called from the sketch.

Instead of playing a file once, it keeps looping it.

The **BOOT0** (`PC_13`) button (right next to the audio jack) is used as a regular pushbutton to loop through the audio files.

The files used are called

`DRUM.wav`, `WARP.wav`,
`BASS.wav` and `SHAKE.wav`. They
are very short (around 1 second).
These needs to be present on the
USB stick used.

You can download them from [this link](#).

```
1  /*
2   * GIGA R1 - Audio P
3   * Simple wav format
4   * In order for this
5   * Furthermore you n
6   */
7
8  #include <Arduino_Ac
9  #include <DigitalOut
10 #include <Arduino_US
11 #include <FATFileSys
12
13 AdvancedDAC dac0(A12
14
15 USBHostMSD msd;
16 mbed::FATFileSystem
17
18 FILE *file = nullptr
19 int sample_size = 0;
20 int samples_count =
21
22 int swapFile;
23
24 void setup() {
25     Serial.begin(11520
26     while (!Serial)
27         ;
28
```

Pulse Density Modulation Support

Pulse Density Support (PDM) is a

represent analog information into digital information; PDM uses a high-frequency stream of 1-bit digital samples. In PDM, a large cluster of ones represents a positive amplitude, while a large cluster of zeros represents a negative amplitude.

The GIGA R1 PDM support can be used with the [built-in PDM library](#). Let's check an interesting example that shows of how to read a PDM microphone wwith the GIGA R1:

```
1  #include <PDM.h>
2
3  // Default number of
4  static const char cha
5
6  // Default PCM output
7  static const int freq
8
9  // Buffer to read sam
10 short sampleBuffer[12
11
12 // Number of audio sa
13 volatile int samplesR
14
15 void setup() {
16   Serial.begin(9600);
17   while (!Serial);
18
19   // Configure the ca
20   PDM.onReceive(onPDM
21   PDM.setGain(30);
22
23   // Initialize PDM m
24   if (!PDM.begin(chan
25       Serial.println("F
26       while (1);
27   }
28 }
```

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here.	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

